

Informe Control 1 - CC4303

Curso: Redes, Primavera 2026.

Institución: Departamento de Ciencias de la Computación, Facultad de Ciencias Físicas y Matemática de la Universidad de Chile.

Profesora: Ivana Bachmann.

Fecha de entrega: 3 de septiembre de 2026.

Integrante: Diego Salinas.

Repositorio de GitHub:

<https://github.com/diego051101/redes-diego-salinas-2026.git>

Declaración de uso de IA/LLM: Se utilizó Claude para resolver dudas de conceptos, comandos e instalación de Linux en Windows, además de guiar de forma general al estudiante en el desarrollo adecuado de cada actividad en momentos severos de bloqueo.

Subsistema de Windows para Linux (WSL)

Requisitos previos:

```
sudo apt update
sudo apt install -y dnsutils
pip3 install dnslib --break-system-packages # o: pip3 install --user dnslib
```

Actividad 1 - Proxy HTTP

Archivos:

- actividad_1/proxy.py
- actividad_1/config.json
- actividad_1/403.jpg

Para correr el proxy:

```
cd actividad_1
python3 proxy.py config.json [puerto] [buffer_size]
```

Por defecto el proxy escucha en 0.0.0.0:8000, lo que permite probarlo tanto para 127.0.0.1 como desde otra máquina de la red. Antes de probarlo en una máquina virtual, reemplazar la constante IP_PROXY dentro de proxy.py por la IP_VM.

Para probarlo con curl en otra terminal:

```
curl example.com
curl example.com -x 127.0.0.1:8000
curl -i cc4303.bachmann.cl/secret -x 127.0.0.1:8000
curl -i cc4303.bachmann.cl -x 127.0.0.1:8000
curl cc4303.bachmann.cl/replace -x 127.0.0.1:8000
```

Para probarlo desde el navegador hace falta configurar un proxy HTTP manual apuntando a 127.0.0.1 a IP_VM con puerto 8000 e ingresar a <http://cc4303.bachmann.cl/>, [/replace](#) y [/secret](#).

Actividad 2 - Resolver DNS

Archivos: actividad_2/resolver.py

Para correr el resolver:

```
cd actividad_2
python3 resolver.py [-debug]
```

El resolver escucha en 0.0.0.0:8000. Antes de probarlo en una máquina virtual, reemplazar la constante IP_SERVIDOR dentro de resolver.py por IP_VM.

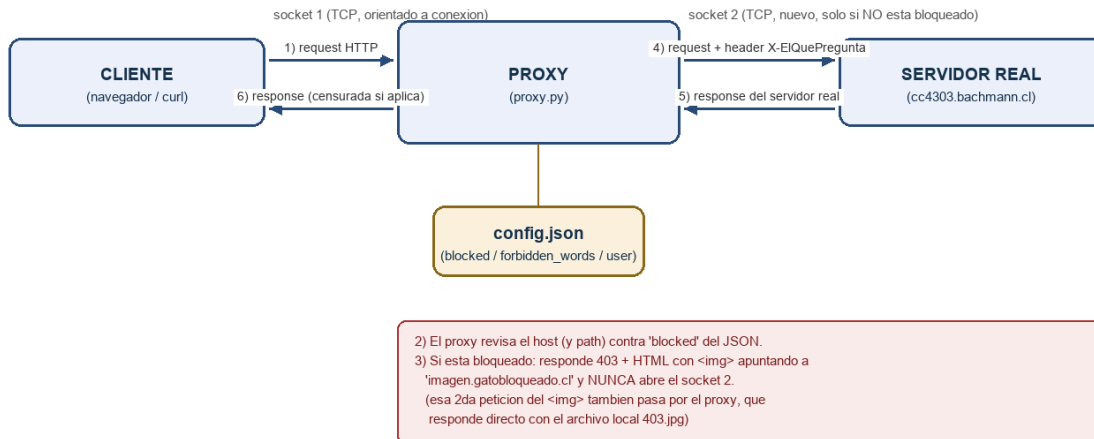
Para probarlo con dig en otra terminal:

```
dig -p8000 @127.0.0.1 example.com
dig -p8000 @127.0.0.1 eol.uchile.cl
dig -p8000 @127.0.0.1 eol.uchile.cl
dig -p8000 @127.0.0.1 www.uchile.cl
dig -p8000 @127.0.0.1 cc4303.bachmann.cl
```

Actividad 1: Proxy HTTP

Flujo de funcionamiento:

Diagrama de flujo del proxy HTTP (actividad 1)



1. El cliente (curl o navegador configurado con proxy) abre una conexión TCP con el proxy y manda una petición HTTP. Al ser una petición dirigida a un proxy, la start line trae la URL completa.
2. El proxy recibe el mensaje completo con `recibir_mensaje_http()` y lo interpreta con `parse_HTTP_message()` para obtener el host, puerto y path necesarios.
3. El proxy revisa si el host está en la lista de bloqueados del JSON. Si está bloqueado, responde directamente con un 403 y cierra la conexión con el cliente, sin abrir ninguna conexión hacia el servidor real.
4. Si no está bloqueado, el proxy abre una SEGUNDA conexión TCP hacia el servidor real, agrega el header con el valor del campo user del JSON y reenvía la petición.
5. El proxy recibe la respuesta del servidor real y si el contenido es de tipo texto, censura las palabras prohibidas del JSON dentro del body recalculando el Content-Length.
6. Finalmente el proxy envía esta respuesta (modificada si corresponde) de vuelta al cliente por la primera conexión y cierra ambas conexiones.

Decisión de diseño: el proxy es de un solo hilo, por lo que atiende una conexión a la vez. Esto sigue la misma estructura del ejemplo `tcp_socket_server.py` visto en clase.

¿Qué extrae `parse_HTTP_message`?

De cada mensaje HTTP se extrae:

- (a) El tipo de mensaje (request o response).
- (b) La start line ya separada en sus partes (método/url/versión o versión/código/texto).
- (c) Los headers guardados como diccionario nombre-valor para leerlos y modificarlos fácilmente.
- (d) El body en bytes, sin decodificar, para no romper contenido binario como imágenes. Esta información es la mínima necesaria para decidir a quién reenviar la petición, si bloquear o no, agregar o leer headers y separar el head del body para censurar texto.

Imagen local y ciclos de comunicación HTTP

Para devolver una imagen local desde el proxy se usa un dominio inventado (`imagen.gatobloqueado.cl`) dentro del `` del HTML de error 403. Como el navegador tiene el proxy configurado, esa segunda petición también pasa por el proxy, que reconoce ese dominio especial y responde directamente con el archivo local `403.jpg`, sin conectarse a ningún servidor real.

¿Cuántos ciclos de comunicación HTTP son necesarios para mostrar una imagen en un navegador?

Se necesitan 2 ciclos: uno para pedir/recibir el HTML de la página y un segundo ciclo para pedir y recibir la imagen en sí.

Manejo de buffers pequeños

- ❖ ¿Cómo sé si llegó el mensaje completo? Cuando llega el separador `\r\n\r\n` del final del HEAD y llegaron tantos bytes de BODY como indica el header Content-Length.
- ❖ ¿Qué pasa si los headers no caben en mi buffer? No es un problema, dado que `recibir_mensaje_http()` sigue llamando a `recv()` dentro de un `while`, acumulando todo lo recibido hasta que ese acumulado contenga `\r\n\r\n`. No se necesita que un solo `recv()` traiga todo el HEAD.
- ❖ ¿Cómo sé que el HEAD llegó completo? Buscando `\r\n\r\n` en todo lo recibido hasta el momento.
- ❖ ¿Y qué ocurre con el BODY? Una vez que el HEAD está completo, se lee Content-Length con `obtener_content_length()` y se siguen pidiendo más datos con `recv()` hasta que el body acumulado alcance esa cantidad de bytes.

Pruebas realizadas y resultados

- ❖ `curl example.com` y `curl example.com -x 127.0.0.1:8000` entregan exactamente el mismo contenido.
- ❖ `curl -i cc4303.bachmann.cl/secret -x 127.0.0.1:8000` responde 403 junto al HTML con la imagen local. Por otra parte `curl -i www.dcc.uchile.cl -x 127.0.0.1:8000` queda bloqueado. Sitios no bloqueados, como `cc4303.bachmann.cl`, siguen funcionando con normalidad.
- ❖ Al pasar por el proxy, `cc4303.bachmann.cl` cambia su mensaje de bienvenida de "Bienvenide ... oh? no puedo ver tu nombre :c!" a "Bienvenide diego.salinas.c@ug.uchile.cl!", confirmando que el header llega correctamente al servidor.
- ❖ `cc4303.bachmann.cl/replace` censura correctamente "proxy" a "[REDACTED]", "DCC" a "[FORBIDDEN]" y "biblioteca" a "[???" sin romper el HTML.
- ❖ Se probó con `buffer_size = 4096, 300` y `20` sobre `/replace`. En los 3 casos el proxy entrega la respuesta completa y correctamente censurada, con el Content-Length recalculado.
- ❖ Se configuró el proxy en las opciones de red del navegador y se confirmó que `http://cc4303.bachmann.cl/secret` ahora muestra el error

403 junto a la imagen local 403.jpg, que `http://cc4303.bachmann.cl/` muestra el mensaje de bienvenida modificado y que `http://cc4303.bachmann.cl/replace` muestra el texto con las palabras censuradas.

Actividad 2: Resolver DNS

Flujo de funcionamiento

1. El resolver abre un socket UDP (no orientado a conexión) y lo asocia a la 2-tupla (IP, 8000).
2. Por cada mensaje recibido de un cliente, lo imprime en bytes y registra el dominio consultado en un historial de las últimas 20 consultas hechas.
3. Si el dominio está entre los tres dominios más repetidos del historial y tiene su IP guardada, el resolver responde directo con esa IP del caché.
4. En caso contrario se llama a la función `resolver()`, que sigue el algoritmo de resolución recursiva visto en el material del curso:
 - ❖ Le pregunta al servidor indicado empezando por el servidor raíz.
 - ❖ Si la respuesta ya trae la IP la retorna.
 - ❖ Si en cambio viene una delegación a otro Name Server, hay dos casos: (1) Si la IP de ese NS vino de regalo en Additional, pregunta ahí directamente. (2) Si no, primero resuelve la IP de ese NS y recién ahí sigue.
 - ❖ Si la respuesta no concuerda con algún caso previo el resolver no sabe qué hacer y la ignora.
5. Si logró resolver, guarda la IP en el caché y responde al cliente.

¿Qué tipo de socket debe usar?

Un socket UDP (`SOCK_DGRAM` no orientado a conexión), ya que así se comunica el DNS. Cada consulta y respuesta es independiente, sin necesidad de mantener un canal abierto.

Caché

Se guardan en `historial_consultas` los últimos 20 dominios preguntados por clientes. Con ese historial se calculan los 3 dominios que más se repiten y solo si un dominio está en ese top 3 y ya se tiene su IP guardada en `ip_guardadas` se responde desde el caché. El modo debug indica explícitamente cuándo se usa el caché.

Prueba: se consultó `eol.uchile.cl` dos veces seguidas. La primera vez el resolver hizo todo el recorrido (raíz a NS de `.cl` a NS de `uchile.cl`); la segunda vez el debug mostró:

```
(debug) 'eol.uchile.cl.' esta en el cache -> IP 146.83.63.64
```

respondiendo de inmediato sin volver a consultar por la red.

Modo debug

A continuación se detalla la salida real obtenida al correr `resolver.py -debug` y consultar `cc4303.bachmann.c`:

```
(debug) Consultando 'cc4303.bachmann.cl.' a '.' con direccion IP '198.41.0.4'
(debug) Consultando 'cc4303.bachmann.cl.' a 'cl2-tld.d-zone.ca.' con direccion IP
'185.159.198.56'
(debug) Consultando 'ns1.digitalocean.com.' a '.' con direccion IP '198.41.0.4'
(debug) Consultando 'ns1.digitalocean.com.' a 'l.gtld-servers.net.' con direccion
IP '192.41.162.30'
(debug) Consultando 'ns1.digitalocean.com.' a 'kim.ns.cloudflare.com.' con
direccion IP '108.162.192.126'
(debug) Consultando 'cc4303.bachmann.cl.' a 'ns1.digitalocean.com.' con direccion
IP '172.64.52.210'
```

Pruebas de funcionalidad

- ❖ `dig -p8000 @IP_VM eol.uchile.cl` responde con IPs de la forma 146.83.63.X. Se encontraron 11 respuestas de tipo A (más 1 CNAME inicial de 12 registros en total en la sección Answer).
- ❖ Repetir la consulta a `eol.uchile.cl` responde con la misma IP (146.83.63.64), pero usando el caché.
- ❖ `dig -p8000 @IP_VM www.uchile.cl` resuelve a 200.89.76.36.
- ❖ `dig -p8000 @IP_VM cc4303.bachmann.cl` resuelve a 104.248.65.245.

Nota: estas pruebas se hicieron con un pequeño script cliente propio basado en dnslib porque el entorno usado para probar no tenía dig instalado (los resultados son equivalentes a los de dig).

Experimentos

1) www.webofscience.com: nuestro resolver no logra resolver este dominio por timeout. Comparando con dig @1.1.1.1 www.webofscience.com, la respuesta real involucra una cadena de CNAME (www.webofscience.com a www-us.webofscience.com a www-us.webofscience.com.cdn.cloudflare.net a IP). Nuestro resolver solo revisa si viene un registro tipo A directo en el Answer o una delegación NS en Authority. Cuando lo que llega es un CNAME puro (sin registro A), cae en el caso (d) y lo ignora. Esto se podría arreglar agregando un caso extra que detecte cuando la respuesta trae un CNAME y que haga una nueva consulta recursiva preguntando por el nombre al que apunta el CNAME, hasta llegar a un tipo A.

2) www.cc4303.bachmann.cl: nuestro resolver tampoco logra resolverlo por timeout. Comparando con dig @1.1.1.1 www.cc4303.bachmann.cl, la respuesta real es NXDOMAIN junto a un registro SOA en Authority. Esto tiene sentido porque solo se configuró cc4303.bachmann.cl como sitio de prueba del curso. Nuestro resolver, al recibir en Authority un registro que no es de tipo NS, no encuentra a quién más preguntarle y cae también en el caso (d). El resultado es el correcto, aunque por una razón distinta a la que muestra dig.

3) Repetir consultas y mirar los Name Servers en modo debug: se repitieron consultas a eol.uchile.cl, www.uchile.cl y cc4303.bachmann.cl y en las 3 el debug mostró que, tras preguntarle a la raíz, el NS de .cl devuelve siempre lo mismo.. En estas pruebas los Name Servers son iguales, porque nuestro resolver siempre le pregunta al mismo servidor raíz y el conjunto de NS de una zona es prácticamente estático. Si se usaran distintas IPs de servidores raíz o si un servidor eligiera al azar entre varias respuestas posibles, ahí sí podría verse variación entre consultas.

Limitaciones conocidas

- ❖ No se siguen cadenas de CNAME.
- ❖ No se distingue explícitamente NXDOMAIN de un resolver que no logra su objetivo (en ambos casos simplemente no se responde nada al cliente).
- ❖ El caché no guarda el TTL real de la respuesta original, solo la IP.
- ❖ No hay protección contra loops de delegación infinitos entre Name Servers.